



Tarea 02

Máquinas de turing

No. de cuenta	Nombre
320148204	González Lucero Alan Uriel
321225087	Rivera Jiménez Isaac
321306102	Tapia Sánchez Jesús



Complejidad Computacional

Índice

1	Ejercicio 0	2
1.1	Respuestas	2
1.2	1	2
1.3	2	2
1.4	3	2
2	Ejercicio 1	3
2.1	Solución	3
2.2	1	3
2.3	Tiempo de ejecución	3
2.4	Ejemplo	3
2.5	3	4
2.6	Tiempo de ejecución	5
2.7	Ejemplo	5
3	Ejercicio 2	6
3.1	Solución al Ejercicio 2.a	6
3.2	Solución al Ejercicio 2.b	15
4	Ejercicio 3	24
4.1	Solución al Ejercicio 3.A	24
4.2	Solución al Ejercicio 3.B	26
4.3	Solución al Ejercicio 3.C	26
5	Ejercicio 4	28
5.1	Solución	28
6	Referencias Bibliográficas	32

Ejercicio 0

Enunciado

Considerando la definición formal vista en clase de la máquina de Turing, responde las siguientes preguntas (justifica ampliamente).

1. ¿Una máquina de Turing podría escribir un símbolo en blanco en la cinta?
2. ¿El alfabeto de la cinta Γ puede ser igual al alfabeto de entrada Σ ?
3. ¿Podemos tener una máquina con un solo estado?

Respuestas

1

Si, pues el símbolo blanco $\sqcup \in \Gamma$. Por lo que la función de transición:

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{\leftarrow, \rightarrow\}$$

puede indicar explícitamente que se reemplazar cualquier carácter leído en la celda por un $\sqcup$, lo que equivale a borrar un símbolo durante el proceso de cómputo.

2

Sea $\Sigma = \{0, 1\}$ el alfabeto de entrada y sea $\Gamma = \{0, 1, \sqcup\}$. Consideremos que la entrada x que recibirá la máquina, es una codificación en binario. Como los elementos de la entrada x , están escritos en la cinta, es claro que $\Sigma \subset \Gamma$. Pero $\Gamma \not\subseteq \Sigma$, pues $\sqcup \notin \Sigma$.

$\therefore \Gamma \neq \Sigma$ no pueden ser iguales. Además, dependiendo el problema a resolver, la cinta puede requerir símbolos adicionales para el procesamiento, que no forman parte de la entrada original.

3

Es posible definirla, pero no sería útil. Pues para que una máquina de Turing realice un cómputo útil o decida un lenguaje, requiere de al menos un estado de rechazo q_r o de aceptación q_a , si la máquina consta del estado inicial: q_0 :

- Si q_0 se define como estado de parada, la máquina se detiene instantáneamente en el paso 0 para cualquier cadena, sin procesar la entrada.
- Si q_0 no es un estado de parada, la máquina nunca podrá cambiar de estado para detenerse, entrando en un bucle infinito para cualquier entrada.

$\therefore$ Una máquina de Turing de un solo estado no puede decidir ningún lenguaje no trivial ni calcular funciones útiles. Por eso en complejidad computacional, se requieren máquinas con al menos dos estados.

Ejercicio 1

Enunciado

Da una descripción general de máquinas de Turing que deciden los siguientes lenguajes y determina el tiempo de ejecución de dichas máquinas.

1. $\{w \in \{a, b\}^* \mid w \text{ contiene un número igual de } a\text{'s y } b\text{'s}\}$
 2. *Opcional* $\{w \in \{0, 1, \#\}^* \mid w \text{ contiene el doble de } 0\text{'s que } 1\text{'s}\}$
 3. $\{w \in \{a, b, c\}^* \mid w \text{ el número de } a\text{'s en } w > \text{número de } b \text{ en } w \geq \text{número de } c \text{ en } w\}$
- Ejemplifica la ejecución (o idea general) de las máquinas propuestas.

Solución

1

1. A la entrada x se le debe de anexar un símbolo inicial $*$ y final $+$, teniendo ahora $x' = *x + \mid x \in \{a, b\}^*$. (Aclarando que $\sqcup$ será ignorado en a la hora de buscar).
2. M lee el símbolo inicial $*$ y se desplaza a la derecha en búsqueda de una a o b . Si se llega inmediatamente a $+$ acepta.
3. Si M lee a imprime $\sqcup$ y procede a buscar b . Si M lee b imprime $\sqcup$ y procede a buscar a . Si se llega a $+$ rechaza.
4. Cuando haya encontrado el símbolo buscado, imprime $\sqcup$ y se desplaza a la izquierda hasta leer $*$
5. Repetir la sucesión de pasos desde 2, hasta rechazar o aceptar.

Tiempo de ejecución

En cada ciclo completo se eliminan exactamente dos caracteres (una a , y una b). Por lo que se realizará un máximo de $n/2$ ciclos antes de terminar.

En el peor de los casos, se debe recorrer toda la cinta hacia la derecha para encontrar la pareja y luego regresar al inicio. Lo que toma un tiempo proporcional a la longitud de la cadena. $O(n)$.

$$\therefore n/2 \times O(n) = O(n^2)$$

Ejemplo

Sea la entrada $x = baba$. Aplicando el primer paso obtenemos la cinta inicial $x' = *baba+$.

1. **Inicio:** $*baba+$
2. **Iteración 1:** Se lee la primera b , se borra y se busca su pareja a a la derecha.
Estado de la cinta tras borrar el primer par (b, a) : $* \sqcup \sqcup ba+$
La cabeza rebobina a la izquierda hasta $*$.

3. **Iteración 2:** Se avanza omitiendo los $\sqcup$, se lee la siguiente b , se borra y se busca su pareja a .
Estado de la cinta tras borrar el segundo par (b, a) : $* \sqcup \sqcup \sqcup \sqcup +$
La cabeza regresa a la izquierda hasta $*$.
4. **Fin:** Se avanza desde $*$, se omiten los $\sqcup$ y se lee el delimitador $+$. La máquina **acepta**.

3

Aclaración general sobre el mecanismo de búsqueda de M :

- **Salto de blancos:** La acción de "*buscar*" un símbolo significa que M se desplaza leyendo la cinta e ignorando las celdas con el símbolo $\sqcup$.
 - **Inercia de movimiento (caso general):** Al leer y borrar un símbolo, M continuará buscando en la ****misma dirección**** en la que se venía desplazando. Si alcanza el delimitador del extremo ($+$ o $*$) sin hallar el objetivo, la máquina invertirá su sentido para realizar un barrido completo hacia el extremo opuesto.
 - **Búsqueda desde un extremo (caso optimizado):** Si el símbolo es borrado justamente en una celda adyacente a un delimitador ($*$ o $+$), la máquina no necesita intentar seguir hacia esa pared; en su lugar, se reorienta inmediatamente para realizar un único recorrido directo de $* \rightarrow +$ (o $+ \rightarrow *$).
1. A la entrada x se le anexan los delimitadores de inicio y fin, teniendo $x' = *x+$ con $x \in \{a, b, c\}^*$.
 2. **Fase 1 (Eliminar tríos c, b, a):** M busca una c . Si la encuentra, la reemplaza por $\sqcup$ y procede a buscar una b :
 - Si encuentra b , la reemplaza por $\sqcup$ y busca una a .
 - Si encuentra a , la reemplaza por $\sqcup$ y **regresa al inicio de la Fase 1**.
 - Si en alguno de los pasos falta la b o la a , M **rechaza**.
 3. **Fase 2 (Eliminar pares b, a):** Cuando ya no queden c 's en la cinta, M busca una b . Si la encuentra, la reemplaza por $\sqcup$ y busca una a :
 - Si encuentra a , la reemplaza por $\sqcup$ y **regresa al inicio de la Fase 2**.
 - Si no encuentra a , M **rechaza** (pues $\#a \leq \#b$).
 4. **Fase 3 (Verificación de a sobrante):** Cuando ya no queden ni c 's ni b 's en la cinta, M busca una a :
 - Si encuentra al menos una a , M **acepta** (garantizando $\#a > \#b \geq \#c$).
 - Si no encuentra ninguna a , M **rechaza**.

Tiempo de ejecución

Sea n la longitud de la cadena de entrada x .

- **Número de iteraciones:** Cada ciclo de la Fase 1 cancela un trío (c, b, a) , la Fase 2 cancela un par (b, a) , y la Fase 3 realiza una verificación final. Por lo tanto, se realizan a lo sumo $O(n)$ iteraciones de borrado.
- **Costo por iteración:** En el peor de los casos, la cabeza de M realiza barridos completos a lo largo de la cinta de longitud $n + 2$ para localizar cada símbolo o verificar los límites $(*$ y $+$). Cada barrido toma tiempo $O(n)$.

$$\text{Tiempo Total} = O(n) \times O(n) = O(n^2)$$

$\therefore$ El tiempo de ejecución de la máquina de Turing M en el peor de los casos es cuadrático, $O(n^2)$.

Ejemplo

Sea la cadena de entrada $w = \text{bcaaaabc}$ con $n = 8$, donde $\#a = 4$, $\#b = 2$, $\#c = 2$. La cinta inicial es $x' = *\text{bcaaaabc}+$.

1. Fase 1 (Cancelación de tríos c, b, a):

- **Ciclo 1:** Se localiza la primera c , la primera b y la primera a , reemplazándolas por $\sqcup$.
Estado de la cinta: $*\sqcup\sqcup\sqcup\text{aaabc}+$
- **Ciclo 2:** Se localiza la segunda c , la b restante y otra a , reemplazándolas por $\sqcup$.
Estado de la cinta: $*\sqcup\sqcup\sqcup\sqcup\text{aa}\sqcup\sqcup+$
- **Fin de Fase 1:** M realiza un barrido completo y no encuentra más símbolos c .

2. Fase 2 (Cancelación de pares b, a):

- M realiza un barrido en la cinta buscando el símbolo b . Dado que no existen más b 's, avanza directamente a la Fase 3.

3. Fase 3 (Verificación de a sobrante):

- M busca el símbolo a y localiza al menos una de las dos a 's restantes en la cinta.
- Al encontrarla, confirma que $\#a > \#b \geq \#c$ ($4 > 2 \geq 2$) y **acepta**.

Ejercicio 2

Enunciado

Para cada uno de los lenguajes, da la descripción completa de una máquina de Turing (incluyendo estados, alfabetos y función de transición) que acepte L , da la función de complejidad de tiempo y ejemplifica la ejecución de la máquina propuesta mostrando las configuraciones que se producen con los ejemplares mencionados.

1. **a** Sea $L = \{0^n 1^m 0^{n+m} \mid n, m \geq 1\}$.

Ejemplares:

(a) 011000

(b) 010

2. **b** Sea $L = \{a\#b \mid a, b \text{ codifican números enteros positivos en binarios y } \max(a, b) = a\}$.

Ejemplares:

(a) $[7]\#[5] \rightarrow 111\#101$

(b) $[2]\#[4] \rightarrow 10\#100$

(c) $[1]\#[1] \rightarrow 1\#1$

3. **OPCIONAL** Sea $L = \{0^n 1^{n^2} \mid n \in \mathbb{N}, n \geq 1\}$

Ejemplares:

(a) 001111

(b) 000111111

Solución al Ejercicio 2.a

1. Descripción Completa de la Máquina de Turing

El lenguaje a decidir es:

$$L = \{0^n 1^m 0^{n+m} \mid n, m \geq 1\}$$

Estrategia Algorítmica General

La máquina opera mediante una estrategia de **cancelación/emparejamiento en dos fases**:

- **Fase 1 (Emparejar 0^n con el primer bloque de 0's del final):** Marca el primer 0 no marcado del inicio con X , avanza a la derecha pasando el bloque 1^m hasta encontrar el primer 0 no marcado del bloque final 0^{n+m} , y lo marca con Z . Regresa a la izquierda al siguiente 0 libre. Repite esto hasta marcar todos los n ceros iniciales como X .
- **Fase 2 (Emparejar 1^m con los 0's restantes del final):** Marca el primer 1 no marcado con Y , avanza a la derecha hasta encontrar el primer 0 no marcado del final y lo marca con Z . Regresa al siguiente 1 libre. Repite esto hasta marcar todos los m unos como Y .
- **Fase 3 (Verificación):** Comprueba que todo el bloque final esté compuesto únicamente por marcas Z y que la cinta termine inmediatamente con el blanco ($\sqcup$).

Especificación Formal (7-Tupla)

Formalmente, definimos la Máquina de Turing M como:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$$

Donde:

- **Conjunto de estados:**

$$Q = \{q_0, q_1, q_{1b}, q_2, q'_0, q_3, q_4, q_5, q'_3, q_6, q_a, q_r\}$$

- q_0, q'_0 : Inicio y bucle de la Fase 1 (procesar 0^n).
- q_1, q_{1b} : Desplazamiento a la derecha buscando el 0 correspondiente en 0^{n+m} .
- q_2 : Retorno a la izquierda tras marcar un 0 con Z .
- q_3, q'_3 : Inicio y bucle de la Fase 2 (procesar 1^m).
- q_4 : Desplazamiento a la derecha buscando el 0 correspondiente en 0^{n+m} .
- q_5 : Retorno a la izquierda tras marcar un 0 con Z .
- q_6 : Fase de verificación final.
- q_a, q_r : Estados de aceptación y rechazo, respectivamente.

- **Alfabeto de entrada:** $\Sigma = \{0, 1\}$

- **Alfabeto de la cinta:** $\Gamma = \{0, 1, X, Y, Z, \sqcup\}$

- X : Marcador para los ceros del primer bloque (0^n).
- Y : Marcador para los unos del bloque medio (1^m).
- Z : Marcador para los ceros del bloque final (0^{n+m}).
- $\sqcup$: Símbolo en blanco.

- **Estado inicial:** q_0

- **Estado de aceptación:** q_a

- **Estado de rechazo:** q_r

Función de Transición $\delta(q, s) = (q', s', m)$

Cualquier transición no indicada explícitamente en la siguiente tabla conduce implícitamente al **estado de rechazo** q_r :

Estado	Leído 0	Leído 1	Símbolo X	Símbolo Y	Símbolo Z	Blanco $\sqcup$
q_0	$(q_1, X, \rightarrow)$	—	—	—	—	—
q_1	$(q_1, 0, \rightarrow)$	$(q_{1b}, 1, \rightarrow)$	—	—	—	—
q_{1b}	$(q_2, Z, \leftarrow)$	$(q_{1b}, 1, \rightarrow)$	—	$(q_{1b}, Y, \rightarrow)$	$(q_{1b}, Z, \rightarrow)$	—
q_2	$(q_2, 0, \leftarrow)$	$(q_2, 1, \leftarrow)$	$(q'_0, X, \rightarrow)$	$(q_2, Y, \leftarrow)$	$(q_2, Z, \leftarrow)$	—
q'_0	$(q_1, X, \rightarrow)$	$(q_3, 1, -)$	—	—	—	—
q_3	—	$(q_4, Y, \rightarrow)$	—	—	—	—
q_4	$(q_5, Z, \leftarrow)$	$(q_4, 1, \rightarrow)$	—	$(q_4, Y, \rightarrow)$	$(q_4, Z, \rightarrow)$	—
q_5	—	$(q_5, 1, \leftarrow)$	—	$(q'_3, Y, \rightarrow)$	$(q_5, Z, \leftarrow)$	—
q'_3	—	$(q_4, Y, \rightarrow)$	—	—	$(q_6, Z, -)$	—
q_6	—	—	—	—	$(q_6, Z, \rightarrow)$	$(q_a, \sqcup, -)$

Funcionamiento de los Estados

Fase 1: Emparejar el primer bloque de ceros (0^n)

- Estado q_0 (Inicio del proceso):**
 Lee el primer símbolo de la entrada. Si es un 0, lo tacha reemplazándolo por X (marcando que ya procesamos el primer 0 de 0^n), se mueve a la derecha y pasa a q_1 . (Si no hay un 0 al inicio, la máquina rechaza inmediatamente).
- Estado q_1 (Cruzar el bloque inicial):**
 Avanza a la derecha atravesando los 0's restantes del primer bloque sin modificarlos, hasta encontrar el primer 1. Al leer el 1, cambia al estado q_{1b} y continúa hacia la derecha.
- Estado q_{1b} (Buscar el 0 final correspondiente):**
 Continúa desplazándose hacia la derecha atravesando los 1's del bloque medio, las marcas Y y las marcas Z previamente colocadas, hasta encontrar el **primer 0 no marcado del bloque final** (0^{n+m}). Al encontrarlo:
 - Lo marca con Z (cancelando un 0 del final).
 - Cambia la dirección hacia la izquierda ($\leftarrow$).
 - Pasa al estado de retorno q_2 .
- Estado q_2 (Rebobinar a la izquierda):**
 Retrocede hacia la izquierda cruzando todos los símbolos ($Z, Y, 1, 0$) sin alterarlos, buscando la última marca X que colocamos al principio. Al leer la X , se mueve un paso a la derecha (sobre el siguiente símbolo a procesar) y pasa a q'_0 .
- Estado q'_0 (Evaluador de la Fase 1):**
 Inspecciona el símbolo que está inmediatamente a la derecha de la última X :

- **Si es un 0:** Significa que aún quedan ceros en el primer bloque 0^n . Lo marca con X , se mueve a la derecha y vuelve a q_1 para repetir la búsqueda de otro 0 final.
- **Si es un 1:** Significa que **ya procesamos todos los n ceros del primer bloque**. Cambia el control a la Fase 2 pasando al estado q_3 (sin modificar el 1).

Fase 2: Emparejar el bloque medio de unos (1^m)

- **Estado q_3 y q'_3 (Procesar un 1 del bloque medio):**
Lee el primer 1 libre del bloque medio. Lo tacha reemplazándolo por Y , se mueve a la derecha y pasa a q_4 .
- **Estado q_4 (Buscar el 0 final para la Y):**
Avanza hacia la derecha saltando los 1's restantes, las Y 's y las Z 's previas, hasta localizar el **siguiente 0 no marcado del bloque final**. Al encontrarlo:
 1. Lo marca con Z .
 2. Cambia la dirección hacia la izquierda ($\leftarrow$).
 3. Pasa a q_5 .
- **Estado q_5 (Rebobinar en la Fase 2):**
Retrocede hacia la izquierda cruzando Z 's y 1's hasta encontrar la última marca Y . Al leer la Y , se mueve un paso a la derecha y pasa a q'_3 .
- **Evaluación en q'_3 :**
Inspecciona el símbolo que sigue a la última Y :
 - **Si es un 1:** Quedan más unos por procesar. Lo marca con Y y pasa a q_4 para cancelar otro 0 final.
 - **Si es una Z :** Significa que **ya procesamos todos los m unos de 1^m** . Pasa a la Fase 3 de verificación en el estado q_6 .

Fase 3: Verificación Final y Aceptación

- **Estado q_6 (Comprobación de limpieza):**
Recorre la cinta hacia la derecha asegurándose de que **todos los símbolos restantes del bloque final sean exclusivamente marcas Z** .
 - Si encuentra un 0 o un 1 sin marcar, la cadena tenía más elementos de los requeridos y la máquina rechaza (q_r).
 - Al encontrar el **símbolo en blanco** ($\sqcup$) inmediatamente después de las Z 's, confirma que la cadena era de la forma exacta $0^n 1^m 0^{n+m}$ y pasa al **estado de aceptación** q_a .

2. Función de Complejidad de Tiempo $f(N)$

Para calcular la función de complejidad temporal en el peor de los casos, analizamos el número exacto de pasos (transiciones) que ejecuta la Máquina de Turing en función del tamaño de la cadena de entrada.

1. Definición del Tamaño de Entrada N

Dada una cadena válida $w = 0^n 1^m 0^{n+m} \in L$:

- El bloque inicial 0^n contiene n **ceros**.
- El bloque medio 1^m contiene m **unos**.
- El bloque final 0^{n+m} contiene $n + m$ **ceros**.

La longitud total de la cadena de entrada es:

$$N = |w| = n + m + (n + m) = 2(n + m)$$

De aquí despejamos la relación fundamental:

$$n + m = \frac{N}{2}$$

2. Conteo de Pasos por Fases

Fase 1: Procesamiento del primer bloque (0^n)

Para cada uno de los n **ceros** del primer bloque:

1. La máquina marca el '0' actual con 'X' y avanza hacia la derecha cruzando los ceros restantes de 0^n , todos los unos de 1^m y las marcas 'Z' previas hasta encontrar el primer '0' libre del bloque final.
 - La distancia recorrida hacia la derecha es exactamente de $n + m$ **celdas**.
2. Al encontrar el '0', lo marca con 'Z' y retrocede hacia la izquierda hasta volver a la última 'X' colocada.
 - La distancia recorrida hacia la izquierda es nuevamente de $n + m$ **celdas**.

Por cada iteración de la Fase 1, la cabeza realiza $2(n + m)$ movimientos. Como hay n ceros en el primer bloque:

$$\text{Pasos en Fase 1} = n \cdot 2(n + m) = 2n(n + m)$$

Fase 2: Procesamiento del bloque medio (1^m)

Para cada uno de los m **unos** del bloque medio:

1. La máquina marca el '1' actual con 'Y' y avanza a la derecha cruzando los unos restantes, las marcas 'Y' y todas las marcas 'Z' previas hasta localizar el siguiente '0' libre del bloque final.

- La distancia recorrida hacia la derecha es de $n + m$ **celdas**.

2. Marca el '0' con 'Z' y retrocede a la izquierda hasta la última 'Y'.

- La distancia recorrida hacia la izquierda es de $n + m$ **celdas**.

Por cada iteración de la Fase 2, la cabeza realiza $2(n + m)$ movimientos. Como hay m unos en el bloque medio:

$$\text{Pasos en Fase 2} = m \cdot 2(n + m) = 2m(n + m)$$

Fase 3: Verificación final y aceptación

Una vez procesados todos los ceros y unos:

1. En el estado q'_3 , al leer la primera 'Z', la máquina pasa a q_6 y recorre todo el bloque final de $n + m$ marcas 'Z' hacia la derecha verificando que no queden ceros libres.

- Requiere $n + m$ **pasos**.

2. Al encontrar el blanco $\sqcup$, lee el símbolo y concluye en el estado de aceptación q_a .

- Requiere **1 paso**.

$$\text{Pasos en Fase 3} = (n + m) + 1$$

3. Expresión Formal de la Función de Tiempo $f(N)$

Sumamos los pasos de las tres fases en términos de los parámetros n y m :

$$T(n, m) = \text{Fase 1} + \text{Fase 2} + \text{Fase 3}$$

$$T(n, m) = 2n(n + m) + 2m(n + m) + (n + m) + 1$$

Factorizamos $(n + m)$:

$$T(n, m) = 2(n + m)(n + m) + (n + m) + 1 = 2(n + m)^2 + (n + m) + 1$$

Ahora sustituimos la relación con el tamaño total de la entrada N , donde $n + m = \frac{N}{2}$:

$$f(N) = 2 \left(\frac{N}{2} \right)^2 + \left(\frac{N}{2} \right) + 1$$

$$f(N) = 2 \left(\frac{N^2}{4} \right) + \frac{N}{2} + 1$$

$$\mathbf{f(N)} = \frac{1}{2}\mathbf{N^2} + \frac{1}{2}\mathbf{N} + \mathbf{1}$$

4. Complejidad Asintótica

Dado que el término dominante es de grado cuadrático, la complejidad temporal de la Máquina de Turing en una sola cinta para este lenguaje es:

$$f(N) = O(N^2)$$

3. Trazas de Ejecución para los Ejemplares Propuestos

A continuación se presentan los cálculos paso a paso representados mediante la secuencia de configuraciones (q, w, u) para la cadena **011000** y para **010**.

Ejemplar 1: Cadena 011000

La cadena 011000 corresponde a $n = 1$ cero inicial, $m = 2$ unos intermedios y $n + m = 3$ ceros finales ($0^11^20^3$). Por lo tanto, **pertenece al lenguaje L** .

Traza de configuraciones paso a paso:

Fase 1: Emparejar el primer bloque de ceros ($n = 1$)

1. $C_1 = (q_0, \epsilon, 011000)$: Estado inicial. Lee el primer 0.
2. $C_2 = (q_1, X, 11000)$: Tacha el 0 con X y avanza a la derecha en q_1 .
3. $C_3 = (q_{1b}, X1, 1000)$: Lee el primer 1 y pasa al estado q_{1b} .
4. $C_4 = (q_{1b}, X11, 000)$: Pasa el segundo 1 y encuentra el primer 0 del bloque final.
5. $C_5 = (q_2, X1, 1Z00)$: Marca el 0 con Z y se mueve a la izquierda en el estado de retorno q_2 .
6. $C_6 = (q_2, X, 11Z00)$: Continúa retrocediendo cruzando el 1.
7. $C_7 = (q_2, \epsilon, X11Z00)$: Alcanza la marca X en el extremo izquierdo.
8. $C_8 = (q'_0, X, 11Z00)$: Lee la X , se mueve un paso a la derecha y evalúa. Al leer un 1, concluye la Fase 1 y pasa a q_3 .

Fase 2: Emparejar el bloque de unos ($m = 2$)

Procesamiento del primer 1:

9. $C_9 = (q_3, X, 11Z00)$: Lee el primer 1.
10. $C_{10} = (q_4, XY, 1Z00)$: Lo tacha con Y y avanza a la derecha en q_4 .
11. $C_{11} = (q_4, XY1, Z00)$: Salta el segundo 1.
12. $C_{12} = (q_4, XY1Z, 00)$: Salta la marca Z .

13. $C_{13} = (q_5, XY1, ZZ0)$: Encuentra el segundo 0 del final, lo marca con Z y retrocede en q_5 .
14. $C_{14} = (q_5, XY, 1ZZ0)$: Retrocede cruzando la Z .
15. $C_{15} = (q_5, X, Y1ZZ0)$: Retrocede cruzando el 1 hasta encontrar la Y .
16. $C_{16} = (q'_3, XY, 1ZZ0)$: Pasa a q'_3 . Como el siguiente símbolo es un 1, continúa la Fase 2.

Procesamiento del segundo 1:

17. $C_{17} = (q_4, XYY, ZZ0)$: Tacha el segundo 1 con Y y avanza a la derecha.
18. $C_{18} = (q_4, XYYZ, Z0)$: Salta la primera Z .
19. $C_{19} = (q_4, XYYZZ, 0)$: Salta la segunda Z .
20. $C_{20} = (q_5, XYYZ, ZZ)$: Encuentra el tercer y último 0 del final, lo marca con Z y retrocede.
21. $C_{21} = (q_5, XYY, ZZZ)$: Retrocede cruzando la Z .
22. $C_{22} = (q_5, XY, YZZZ)$: Retrocede cruzando la segunda Z hasta llegar a la última Y .
23. $C_{23} = (q'_3, XYY, ZZZ)$: Pasa a q'_3 . Lee una Z (ya no quedan unos), lo que confirma el fin de la Fase 2 y cambia a q_6 .

Fase 3: Verificación final y Aceptación

24. $C_{24} = (q_6, XYY, ZZZ)$: En q_6 , verifica la primera Z .
25. $C_{25} = (q_6, XYYZ, ZZ)$: Verifica la segunda Z .
26. $C_{26} = (q_6, XYYZZ, Z)$: Verifica la tercera Z .
27. $C_{27} = (q_6, XYYZZZ, \sqcup)$: Encuentra el símbolo blanco $\sqcup$ al final de la cinta.
28. $C_{28} = (q_a, XYYZZZ, \sqcup)$: Alcanza el **estado de aceptación** q_a y se detiene.

Resultado: La cadena 011000 es **ACEPTADA**.

Ejemplar 2: Cadena 010

La cadena 010 tiene $n = 1$ cero inicial, $m = 1$ uno medio, pero únicamente **1 cero final** en lugar de los $n + m = 2$ ceros requeridos ($0^11^10^1$). Por lo tanto, **no pertenece al lenguaje** L .

Traza de configuraciones paso a paso:

1. $C_1 = (q_0, \epsilon, 010)$: Estado inicial. Lee el primer 0.
2. $C_2 = (q_1, X, 10)$: Tacha el 0 con X y avanza en q_1 .

3. $C_3 = (q_{1b}, X1, 0)$: Lee el 1 y pasa al estado q_{1b} .
4. $C_4 = (q_2, X, 1Z)$: Encuentra el único 0 final, lo marca con Z y retrocede en q_2 .
5. $C_5 = (q_2, \epsilon, X1Z)$: Retrocede cruzando el 1 hasta llegar a la X .
6. $C_6 = (q'_0, X, 1Z)$: Lee la X y pasa a q'_0 . Al leer el 1, concluye la Fase 1 y pasa a q_3 .
7. $C_7 = (q_3, X, 1Z)$: En q_3 , lee el único 1.
8. $C_8 = (q_4, XY, Z)$: Tacha el 1 con Y y avanza a la derecha buscando un 0 final sin marcar.
9. $C_9 = (q_4, XYZ, \sqcup)$: Salta la marca Z y encuentra el símbolo en blanco $\sqcup$ **sin haber encontrado el 0 que requería para emparejar la Y .**
10. $C_{10} = (q_r, XYZ, \sqcup)$: Dado que no existe una transición explícita para $\delta(q_4, \sqcup)$, la máquina ejecuta la regla implícita de fallo y pasa al **estado de rechazo** q_r .

Resultado: La cadena **010** es **RECHAZADA**.

Solución al Ejercicio 2.b

1. Descripción Completa de la Máquina de Turing

El lenguaje a decidir es:

$$L = \{a\#b \mid a, b \in \{0, 1\}^+ \text{ en binario y } \max(a, b) = a\}$$

Es decir, la máquina debe aceptar la cadena si y solo si el número binario a es **mayor o igual** que el número binario b ($a \geq b$).

Estrategia Algorítmica General

Para comparar dos números enteros positivos en binario a y b , la máquina realiza un **emparejamiento bit a bit de izquierda a derecha (del bit más significativo al menos significativo)** registrando dos aspectos simultáneamente:

1. **Comparación de Longitud ($|a|$ vs $|b|$):** Empareja cada bit de a con un bit de b .
 - Si a tiene más bits que b ($|a| > |b|$), entonces $a > b \implies$ **Acepta**.
 - Si b tiene más bits que a ($|a| < |b|$), entonces $a < b \implies$ **Rechaza**.
2. **Comparación de Valor (si $|a| = |b|$):** Mientras empareja los bits, la máquina guarda en su memoria de estados el resultado del **primer bit en el que a y b difieren**:
 - Si el primer bit donde difieren tiene un 1 en a y un 0 en b , la relación es **MAYOR** (G).
 - Si tiene un 0 en a y un 1 en b , la relación es **MENOR** (L).
 - Si todos los bits son idénticos, la relación es **IGUAL** (E).

Al finalizar la lectura de ambas cadenas, si las longitudes son iguales ($|a| = |b|$), la máquina acepta si el estado registrado es **MAYOR** (G) o **IGUAL** (E), y rechaza si es **MENOR** (L).

Especificación Formal (7-Tupla)

Formalmente, definimos la Máquina de Turing M como:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$$

Donde:

- **Conjunto de estados:**

$$Q = \{q_{0,E}, q_{0,G}, q_{0,L}, q_{r0,E}, q_{r0,G}, q_{r0,L}, q_{r1,E}, q_{r1,G}, q_{r1,L}, q_{ret,E}, q_{ret,G}, q_{ret,L}, q_{check,E}, q_{check,G}, q_{check,L}, q_a, q_r\}$$

- El subíndice E, G, L en cada estado representa la memoria del estado de comparación hasta el momento:

- * **E (Equal):** Todos los bits procesados hasta ahora son iguales ($a = b$).
- * **G (Greater):** La primera diferencia encontrada fue 1 en a y 0 en b ($a > b$).
- * **L (Less):** La primera diferencia encontrada fue 0 en a y 1 en b ($a < b$).
- **Alfabeto de entrada:** $\Sigma = \{0, 1, \#\}$
- **Alfabeto de la cinta:** $\Gamma = \{0, 1, \#, X, Y, \sqcup\}$
 - X : Marcador para los bits ya procesados en la cadena a .
 - Y : Marcador para los bits ya procesados en la cadena b .
 - $\sqcup$: Símbolo en blanco.
- **Estado inicial:** $q_0 = q_{0,E}$ (al inicio asumimos que son iguales hasta hallar la primera diferencia).
- **Estado de aceptación:** q_a
- **Estado de rechazo:** q_r

Función de Transición $\delta(q, s) = (q', s', m)$

Cualquier transición no indicada explícitamente en la siguiente tabla conduce implícitamente al **estado de rechazo** q_r :

Estado Actual (q)	Leído 0	Leído 1	Símbolo #	Símbolo X	Símbolo Y	Blanco $\sqcup$
$q_{0,K}$ ($K \in \{E, G, L\}$)	$(q_{r0,K}, X, \rightarrow)$	$(q_{r1,K}, X, \rightarrow)$	$(q_{check,K}, \#, \rightarrow)$	—	—	—
$q_{r0,K}$ (al leer bit en b)	$(q_{r0,K}, 0, \rightarrow)$ $\rightarrow$ en b : $(q_{ret,K}, Y, \leftarrow)$	$(q_{r0,K}, 1, \rightarrow)$ $\rightarrow$ en b : $(q_{ret,K'}, Y, \leftarrow)$	$(q_{r0,K}, \#, \rightarrow)$	—	$(q_{r0,K}, Y, \rightarrow)$	$(q_a, \sqcup, -)$
$q_{r1,K}$ (al leer bit en b)	$(q_{r1,K}, 0, \rightarrow)$ $\rightarrow$ en b : $(q_{ret,K''}, Y, \leftarrow)$	$(q_{r1,K}, 1, \rightarrow)$ $\rightarrow$ en b : $(q_{ret,K}, Y, \leftarrow)$	$(q_{r1,K}, \#, \rightarrow)$	—	$(q_{r1,K}, Y, \rightarrow)$	$(q_a, \sqcup, -)$
$q_{ret,K}$	$(q_{ret,K}, 0, \leftarrow)$	$(q_{ret,K}, 1, \leftarrow)$	$(q_{ret,K}, \#, \leftarrow)$	$(q_{0,K}, X, \rightarrow)$	$(q_{ret,K}, Y, \leftarrow)$	—
$q_{check,K}$	—	—	—	—	$(q_{check,K}, Y, \rightarrow)$	(ver abajo)

- **Actualización de la memoria de comparación:**
 - En $q_{r0,K}$ al leer 1 en b (0 vs 1): Si $K = E \implies K' = L$; si $K = G \implies K' = G$; si $K = L \implies K' = L$.
 - En $q_{r1,K}$ al leer 0 en b (1 vs 0): Si $K = E \implies K'' = G$; si $K = G \implies K'' = G$; si $K = L \implies K'' = L$.
- **Regla de terminación en $q_{check,K}$ al leer $\sqcup$:**
 - Si $K = E$ o $K = G \implies (q_a, \sqcup, -)$ (**Acepta**).

- Si $K = L \implies (q_r, \sqcup, -)$ (**Rechaza**).
- *Nota:* Si $q_{check,K}$ lee 0 o 1 en lugar de $\sqcup$, significa que $|a| < |b|$, por lo que pasa a q_r (**Rechaza**).

Funcionamiento de los Estados

- **Estados $q_{0,K}$ (Lectura del siguiente bit de a):**
Ubicados en la cadena a , leen el primer bit no procesado.
 - Si leen 0 o 1, lo marcan con X , recuerdan en el estado qué bit leyeron ($r0$ o $r1$) y avanzan a la derecha.
 - Si leen $\#$, significa que **la cadena a ya no tiene más bits**. Pasan al estado $q_{check,K}$ para comprobar el final de b .
- **Estados $q_{r0,K}$ y $q_{r1,K}$ (Buscar el bit correspondiente en b):**
Avanzan hacia la derecha cruzando el resto de a , el separador $\#$ y las marcas Y anteriores hasta encontrar el **primer bit libre de b** :
 - **Si encuentran el blanco $\sqcup$:** Significa que a aún tenía bits pero b ya se terminó ($|a| > |b|$). Por lo tanto, $a > b$ y la máquina **acepta inmediatamente** (q_a).
 - **Si encuentran un bit en b (0 o 1):** Comparan el bit que traían de a con el bit leído en b . Si es la primera diferencia, actualizan la memoria del estado ($E \rightarrow G$ o $E \rightarrow L$). Marcan el bit de b con Y y cambian a los estados de retorno $q_{ret,K}$.
- **Estados $q_{ret,K}$ (Rebobinar a la izquierda):**
Retroceden hacia la izquierda cruzando todos los símbolos (Y , $\#$, 1, 0) sin modificarlos hasta encontrar la última marca X en a . Al leer X , se mueven un paso a la derecha sobre el siguiente bit de a y vuelven a $q_{0,K}$.
- **Estados $q_{check,K}$ (Verificación de longitud y resultado final):**
Se alcanzan cuando a se quedó sin bits. Recorren las marcas Y de b hacia la derecha:
 - **Si leen un 0 o 1 sin marcar:** Significa que b tiene más bits que a ($|a| < |b|$), por lo que $a < b$ y la máquina **rechaza** (q_r).
 - **Si leen el blanco $\sqcup$:** Significa que b también se quedó sin bits exactamente al mismo tiempo ($|a| = |b|$).
 - * Si la memoria conservó E ($a = b$) o G ($a > b$), pasa al estado de **aceptación** q_a .
 - * Si la memoria conservó L ($a < b$), pasa al estado de **rechazo** q_r .

2. Función de Complejidad de Tiempo $f(N)$

Para calcular la función de complejidad temporal en el peor de los casos para la máquina que compara dos números binarios a y b ($w = a\#b$), analizamos el número de transiciones que ejecuta en función del tamaño total de la entrada N .

1. Definición de las Variables de Entrada

Dada la cadena de entrada $w = a\#b$:

- Sea $n = |a|$ la cantidad de bits del número a .
- Sea $m = |b|$ la cantidad de bits del número b .
- El separador $\#$ ocupa **1 celda**.

La longitud total de la cadena de entrada es:

$$N = |w| = n + m + 1$$

De aquí despejamos la suma de las longitudes de ambos números binarios:

$$n + m = N - 1$$

2. Conteo de Pasos por Iteración (Emparejamiento de Bits)

Para cada bit i de la cadena a (con $i = 1, 2, \dots, \min(n, m)$):

1. Desplazamiento a la derecha (de a hacia b):

- La cabeza se encuentra en la posición del i -ésimo bit de a . Lo marca con X y avanza hacia la derecha atravesando los $(n - i)$ bits restantes de a , la celda del separador $\#$ y las $(i - 1)$ marcas Y de b hasta posicionarse sobre el i -ésimo bit de b .
- La distancia exacta recorrida a la derecha es:

$$(n - i) + 1 + (i - 1) + 1 = n + 1 \text{ celdas}$$

2. Desplazamiento a la izquierda (retorno de b hacia a):

- En el i -ésimo bit de b , lo marca con Y y retrocede hacia la izquierda cruzando las mismas $n + 1$ celdas hasta volver a la marca X en a , dando **1 paso adicional a la derecha** para ubicarse sobre el siguiente bit libre ($(i + 1)$ -ésimo bit) de a .
- La distancia recorrida de regreso es:

$$(n + 1) \text{ celdas hacia la izquierda} + 1 \text{ paso a la derecha} = n + 2 \text{ celdas}$$

Pasos totales por cada pareja de bits procesada:

$$(n + 1) \text{ (ida)} + (n + 1) \text{ (vuelta)} = 2(n + 1) \text{ pasos}$$

3. Análisis por Casos de Longitud

- **Caso A:** $|a| > |b|$ ($n > m$):
 - Empareja los primeros m bits de ambas cadenas: $m \cdot 2(n + 1)$ pasos.
 - En el bit $(m + 1)$ -ésimo de a , avanza $n + 1$ celdas a la derecha buscando un bit en b , pero encuentra el blanco $\sqcup$ (indicando que b ya terminó). Acepta de inmediato.
 - Pasos totales: $2m(n + 1) + (n + 2)$.
- **Caso B:** $|a| \leq |b|$ ($n \leq m$):
 - Empareja los n bits de a : $n \cdot 2(n + 1) = 2n(n + 1)$ pasos.
 - Tras procesar el n -ésimo bit de a , la cabeza llega al separador $\#$, cambia al estado q_{check} y avanza a la derecha cruzando las n marcas Y de b (n pasos) y lee el siguiente símbolo ($\sqcup$ si $n = m$, o un bit libre si $m > n$) para decidir en 1 paso adicional.
 - Pasos totales: $2n(n + 1) + (n + 1) = (2n + 1)(n + 1)$ pasos.

4. Deducción del Peor Caso en Términos de N

El **peor de los casos** (máximo número de transiciones) ocurre cuando a y b **tienen exactamente la misma longitud** ($n = m$), ya que la máquina se ve obligada a realizar el recorrido completo de ida y vuelta para todos los bits de a y b , más la verificación final completa de la cadena antes de aceptar o rechazar.

Si $n = m$:

1. Sustituimos $n = m$ en la fórmula de longitud total:

$$N = n + n + 1 = 2n + 1 \implies n = \frac{N - 1}{2}$$

2. Sustituimos $n = \frac{N-1}{2}$ en la función de pasos del peor caso $T(n) = (2n + 1)(n + 1)$:

$$f(N) = \left(2 \left(\frac{N-1}{2}\right) + 1\right) \cdot \left(\frac{N-1}{2} + 1\right)$$

$$f(N) = (N - 1 + 1) \cdot \left(\frac{N - 1 + 2}{2}\right)$$

$$f(N) = N \cdot \left(\frac{N + 1}{2}\right)$$

$$f(N) = \frac{1}{2}N^2 + \frac{1}{2}N$$

5. Complejidad Asintótica

Dado que el término dominante es de grado cuadrático, la complejidad temporal en el peor de los casos es:

$$f(N) = \mathcal{O}(N^2)$$

3. Trazas de Ejecución para los Ejemplares Propuestos

A continuación se presentan las trazas formales de cómputo paso a paso mediante la secuencia de configuraciones (q, w, u) para los tres ejemplares propuestos: **111#101**, **10#100** y **1#1**.

Ejemplar 1: Cadena 111#101

- **Entrada:** $a = 111_2$ (7_{10}) y $b = 101_2$ (5_{10}).
- **Condición:** $a \geq b$ ($7 \geq 5$).
- **Resultado esperado:** ACEPTADA.

Traza de configuraciones paso a paso:

Comparación del primer bit: 1 de a vs 1 de b

1. $C_1 = (q_{0,E}, \epsilon, 111\#101)$: Inicio en estado de igualdad (E). Lee el primer 1 de a .
2. $C_2 = (q_{r1,E}, X, 11\#101)$: Tacha con X y avanza a la derecha recordando que leyó un 1.
3. $C_3 = (q_{r1,E}, X1, 1\#101)$: Salta el segundo 1.
4. $C_4 = (q_{r1,E}, X11, \#101)$: Salta el tercer 1.
5. $C_5 = (q_{r1,E}, X11\#, 101)$: Salta el separador $\#$ y encuentra el primer bit de b (1). Como trae un 1 de a y lee un 1 en b , la memoria se mantiene en **igualdad** (E). Lo tacha con Y y retrocede en $q_{ret,E}$.
6. $C_6 = (q_{ret,E}, X11, \#Y01)$: Retrocede cruzando $\#$.
7. $C_7 = (q_{ret,E}, X1, 1\#Y01)$: Retrocede cruzando el último 1.
8. $C_8 = (q_{ret,E}, X, 11\#Y01)$: Retrocede cruzando el penúltimo 1.
9. $C_9 = (q_{ret,E}, \epsilon, X11\#Y01)$: Lee X , se mueve un paso a la derecha sobre el segundo 1 de a y pasa a $q_{0,E}$.

Comparación del segundo bit: 1 de a vs 0 de b

10. $C_{10} = (q_{0,E}, X, 11\#Y01)$: Lee el segundo 1 de a .
11. $C_{11} = (q_{r1,E}, XX, 1\#Y01)$: Tacha con X y avanza a la derecha.
12. $C_{12} = (q_{r1,E}, XX1, \#Y01)$: Salta el último 1.
13. $C_{13} = (q_{r1,E}, XX1\#, Y01)$: Salta la marca $\#$.
14. $C_{14} = (q_{r1,E}, XX1\#Y, 01)$: Salta la primera Y y encuentra el segundo bit de b (0). Como trae un 1 de a y lee un 0 en b ($1 > 0$), **la memoria cambia de $E \rightarrow G$ (MAYOR)**. Lo tacha con Y y retrocede en $q_{ret,G}$.
15. $C_{15} = (q_{ret,G}, XX1\#, YY1)$: Retrocede en estado G .

16. $C_{16} = (q_{ret,G}, XX1, \#YY1)$: Retrocede cruzando $\#$.
17. $C_{17} = (q_{ret,G}, XX, 1\#YY1)$: Retrocede cruzando el 1.
18. $C_{18} = (q_{ret,G}, X, X1\#YY1)$: Lee X , se mueve un paso a la derecha sobre el tercer 1 de a y pasa a $q_{0,G}$.

Comparación del tercer bit: 1 de a vs 1 de b

19. $C_{19} = (q_{0,G}, XX, 1\#YY1)$: Lee el tercer 1 de a .
20. $C_{20} = (q_{r1,G}, XXX, \#YY1)$: Tacha con X y avanza a la derecha en memoria G .
21. $C_{21} = (q_{r1,G}, XXX\#, YY1)$: Salta $\#$.
22. $C_{22} = (q_{r1,G}, XXX\#Y, Y1)$: Salta la primera Y .
23. $C_{23} = (q_{r1,G}, XXX\#YY, 1)$: Salta la segunda Y y lee el último bit de b (1). Como la memoria ya estaba en G , se conserva en G (**MAYOR**). Tacha con Y y retrocede en $q_{ret,G}$.
24. $C_{24} = (q_{ret,G}, XXX\#Y, YY)$: Retrocede cruzando Y .
25. $C_{25} = (q_{ret,G}, XXX\#, YYY)$: Retrocede cruzando Y .
26. $C_{26} = (q_{ret,G}, XX, X\#YYY)$: Retrocede cruzando $\#$ y lee X , moviéndose un paso a la derecha sobre la $\#$. Pasa a $q_{0,G}$.

Verificación de fin de cadena y Aceptación

27. $C_{27} = (q_{0,G}, XXX, \#YYY)$: Lee $\#$ (confirmando que a no tiene más bits). Pasa al estado $q_{check,G}$ y avanza a la derecha.
28. $C_{28} = (q_{check,G}, XXX\#, YYY)$: Salta la primera Y .
29. $C_{29} = (q_{check,G}, XXX\#Y, YY)$: Salta la segunda Y .
30. $C_{30} = (q_{check,G}, XXX\#YY, Y)$: Salta la tercera Y .
31. $C_{31} = (q_{check,G}, XXX\#YYY, \sqcup)$: Encuentra el símbolo en blanco $\sqcup$ al final de la cinta (confirmando que $|a| = |b|$).
32. $C_{32} = (q_a, XXX\#YYY, \sqcup)$: Como la memoria conservó el estado G ($a > b$), pasa al **estado de aceptación** q_a y se detiene.

Resultado: La cadena **111#101** es **ACEPTADA**.

Ejemplar 2: Cadena 10#100

- **Entrada:** $a = 10_2$ (2_{10} , $|a| = 2$) y $b = 100_2$ (4_{10} , $|b| = 3$).
- **Condición:** $a < b$ (longitud de a menor que la de b).

- **Resultado esperado: RECHAZADA.**

Traza de configuraciones paso a paso:

1. $C_1 = (q_{0,E}, \epsilon, 10\#100)$: Lee el primer 1 de a .
2. $C_2 = (q_{r1,E}, X, 0\#100)$: Tacha con X y avanza a la derecha.
3. $C_3 = (q_{r1,E}, X0, \#100)$: Salta el 0.
4. $C_4 = (q_{r1,E}, X0\#, 100)$: Salta $\#$ y lee el primer bit de b (1). Como 1 vs $1 \implies E$, tacha con Y y retrocede en $q_{ret,E}$.
5. $C_5 = (q_{ret,E}, X0, \#Y00)$: Retrocede cruzando $\#$.
6. $C_6 = (q_{ret,E}, X, 0\#Y00)$: Retrocede cruzando 0.
7. $C_7 = (q_{ret,E}, \epsilon, X0\#Y00)$: Lee X , avanza a la derecha sobre el 0 de a y pasa a $q_{0,E}$.
8. $C_8 = (q_{0,E}, X, 0\#Y00)$: Lee el 0 de a .
9. $C_9 = (q_{r0,E}, XX, \#Y00)$: Tacha con X y avanza a la derecha.
10. $C_{10} = (q_{r0,E}, XX\#, Y00)$: Salta $\#$.
11. $C_{11} = (q_{r0,E}, XX\#Y, 00)$: Salta Y y lee el segundo bit de b (0). Como 0 vs $0 \implies E$, tacha con Y y retrocede en $q_{ret,E}$.
12. $C_{12} = (q_{ret,E}, XX\#, YY0)$: Retrocede cruzando Y .
13. $C_{13} = (q_{ret,E}, X, X\#YY0)$: Retrocede cruzando $\#$ y lee X , moviéndose a la derecha sobre la $\#$ en estado $q_{0,E}$.
14. $C_{14} = (q_{0,E}, XX, \#YY0)$: Lee $\#$ (cadena a terminada). Pasa a $q_{check,E}$ y avanza a la derecha.
15. $C_{15} = (q_{check,E}, XX\#, YY0)$: Salta la primera Y .
16. $C_{16} = (q_{check,E}, XX\#Y, Y0)$: Salta la segunda Y .
17. $C_{17} = (q_{check,E}, XX\#YY, 0)$: **Encuentra un bit 0 no procesado en b** (lo que demuestra que $|a| < |b|$).
18. $C_{18} = (q_r, XX\#YY, 0)$: Al no poder completar la verificación de longitud, pasa al **estado de rechazo** q_r y se detiene.

Resultado: La cadena $10\#100$ es **RECHAZADA**.

Ejemplar 3: Cadena $1\#1$

- **Entrada:** $a = 1_2$ (1_{10}) y $b = 1_2$ (1_{10}).
- **Condición:** $a = b$ ($1 \geq 1$).

- **Resultado esperado: ACEPTADA.**

Traza de configuraciones paso a paso:

1. $C_1 = (q_{0,E}, \epsilon, 1\#1)$: Lee el único 1 de a .
2. $C_2 = (q_{r1,E}, X, \#1)$: Tacha con X y avanza a la derecha.
3. $C_3 = (q_{r1,E}, X\#, 1)$: Salta $\#$ y lee el único bit de b (1). Como $1 \text{ vs } 1 \implies E$, tacha con Y y retrocede en $q_{ret,E}$.
4. $C_4 = (q_{ret,E}, X, \#Y)$: Retrocede cruzando $\#$.
5. $C_5 = (q_{ret,E}, \epsilon, X\#Y)$: Lee X , se mueve a la derecha sobre la $\#$ y pasa a $q_{0,E}$.
6. $C_6 = (q_{0,E}, X, \#Y)$: Lee $\#$ (cadena a terminada). Pasa a $q_{check,E}$ y avanza a la derecha.
7. $C_7 = (q_{check,E}, X\#, Y)$: Salta la Y .
8. $C_8 = (q_{check,E}, X\#Y, \sqcup)$: Encuentra el símbolo blanco $\sqcup$ al final de la cinta (confirmando $|a| = |b|$).
9. $C_9 = (q_a, X\#Y, \sqcup)$: Como la memoria conservó el estado E ($a = b$), pasa al **estado de aceptación** q_a y se detiene.

Resultado: La cadena $1\#1$ es **ACEPTADA**.

Ejercicio 3

Enunciado

Considera la Máquina de Turing definida por:

$$M = (\{q_0, q_1, q_2, q_a, q_r\}, \{0, 1\}, \{0, 1, \sqcup\}, \delta, q_0, q_a, q_r)$$

Describe el Lenguaje L_M para cada una de las siguientes definiciones de δ . Para cada inciso:

- Justifica tu respuesta agregando un par de ejecuciones (al menos una de aceptación y una de rechazo) para cada inciso.
- Da la función $f_M(N)$ del tiempo de ejecución de la máquina con respecto a la longitud N de la cadena.

1. **A.**

- (a) $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
- (b) $\delta(q_1, 1) = (q_2, 0, \leftarrow)$
- (c) $\delta(q_2, 1) = (q_0, 1, \rightarrow)$
- (d) $\delta(q_1, \sqcup) = (q_a, \sqcup, \rightarrow)$

2. **B.**

- (a) $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
- (b) $\delta(q_1, 1) = (q_0, 0, \rightarrow)$
- (c) $\delta(q_1, \sqcup) = (q_a, \sqcup, \leftarrow)$

3. **C.**

- (a) $\delta(q_0, 0) = (q_0, \sqcup, \rightarrow)$
- (b) $\delta(q_0, 1) = (q_1, \sqcup, \rightarrow)$
- (c) $\delta(q_1, 1) = (q_1, \sqcup, \rightarrow)$
- (d) $\delta(q_1, \sqcup) = (q_a, \sqcup, \leftarrow)$

Considera que las reglas (transiciones) que no se indican de manera explícita llevan al estado de rechazo q_r .

Solución al Ejercicio 3.A

Lenguaje descrito:

$$L_M = \{01^m \mid m \geq 0\}$$

El estado inicial q_0 exige que el primer símbolo sea forzosamente un '0'. Al leerlo, escribe un '1' y pasa a q_1 . Si la cadena tiene '1's posteriores, la máquina entra en un ciclo de tres pasos ($q_1 \rightarrow q_2 \rightarrow q_0 \rightarrow q_1$) cuyo único propósito es avanzar el cabezal hacia la derecha dejando un '1' escrito, funcionando como un mecanismo complejo para saltar los '1's. Acepta si, estando en q_1 , encuentra el final de la cadena ($\sqcup$). Si en q_1 encuentra un '0', rechaza.

Consideramos las cadenas originales para ejecución: $w_r = 011011$ (rechazo) y $w_a = 011111$ (aceptación).

Ejecución de rechazo para $w_r = 011011$:

$C_0 = (q_0, \epsilon, 011011)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_1 = (q_1, 1, 11011)$	Aplica $\delta(q_1, 1) = (q_2, 0, \leftarrow)$
$C_2 = (q_2, \epsilon, 101011)$	Aplica $\delta(q_2, 1) = (q_0, 1, \rightarrow)$
$C_3 = (q_0, 1, 01011)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_4 = (q_1, 11, 1011)$	Aplica $\delta(q_1, 1) = (q_2, 0, \leftarrow)$
$C_5 = (q_2, 1, 10011)$	Aplica $\delta(q_2, 1) = (q_0, 1, \rightarrow)$
$C_6 = (q_0, 11, 0011)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_7 = (q_1, 111, 011)$	Sin transición definida para $\delta(q_1, 0)$. RECHAZA (q_r) .

Ejecución de aceptación para $w_a = 011111$:

$C_0 = (q_0, \epsilon, 011111)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_1 = (q_1, 1, 11111)$	Aplica $\delta(q_1, 1) = (q_2, 0, \leftarrow)$
$C_2 = (q_2, \epsilon, 101111)$	Aplica $\delta(q_2, 1) = (q_0, 1, \rightarrow)$
$C_3 = (q_0, 1, 01111)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_4 = (q_1, 11, 1111)$	Aplica $\delta(q_1, 1) = (q_2, 0, \leftarrow)$
$C_5 = (q_2, 1, 10111)$	Aplica $\delta(q_2, 1) = (q_0, 1, \rightarrow)$
$C_6 = (q_0, 11, 0111)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_7 = (q_1, 111, 111)$	Aplica $\delta(q_1, 1) = (q_2, 0, \leftarrow)$
$C_8 = (q_2, 11, 1011)$	Aplica $\delta(q_2, 1) = (q_0, 1, \rightarrow)$
$C_9 = (q_0, 111, 011)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_{10} = (q_1, 1111, 11)$	Aplica $\delta(q_1, 1) = (q_2, 0, \leftarrow)$
$C_{11} = (q_2, 111, 101)$	Aplica $\delta(q_2, 1) = (q_0, 1, \rightarrow)$
$C_{12} = (q_0, 1111, 01)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_{13} = (q_1, 11111, 1)$	Aplica $\delta(q_1, 1) = (q_2, 0, \leftarrow)$
$C_{14} = (q_2, 1111, 10)$	Aplica $\delta(q_2, 1) = (q_0, 1, \rightarrow)$
$C_{15} = (q_0, 11111, 0)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_{16} = (q_1, 111111, \sqcup)$	Aplica $\delta(q_1, \sqcup) = (q_a, \sqcup, \rightarrow)$
$C_{17} = (q_a, 111111\sqcup, \epsilon)$	ACEPTA.

Complejidad de M : Para una cadena válida de longitud $N = m + 1$, la máquina emplea 1 transición para leer el primer '0', realiza un ciclo de 3 transiciones por cada uno de los m unos, y 1 transición para aceptar con el $\sqcup$. El número total de transiciones es $1 + 3m + 1 = 3m + 2$. Sustituyendo $m = N - 1$, obtenemos $3(N - 1) + 2 = 3N - 1$ pasos. Por lo tanto:

$$f_M(N) = 3N - 1 \implies \mathbf{f}_M(\mathbf{N}) \in \mathcal{O}(\mathbf{N})$$

Solución al Ejercicio 3.B

Lenguaje descrito:

$$L_M = \{0(10)^m \mid m \geq 0\}$$

El autómata alterna estrictamente sus estados: en q_0 espera obligatoriamente un '0' y pasa a q_1 ; en q_1 espera obligatoriamente un '1' y regresa a q_0 . La aceptación solo ocurre si la máquina lee el blanco ($\sqcup$) estando en q_1 , lo que significa que el último símbolo procesado debió ser un '0'. Por ende, las cadenas válidas deben iniciar en '0' y alternar entre '1' y '0', terminando siempre en '0'.

Consideramos las cadenas originales para ejecución: $w_r = 011011$ (rechazo) y $w_a = 01010$ (aceptación).

Ejecución de rechazo para $w_r = 011011$:

$C_0 = (q_0, \epsilon, 011011)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_1 = (q_1, 1, 11011)$	Aplica $\delta(q_1, 1) = (q_0, 0, \rightarrow)$
$C_2 = (q_0, 10, 1011)$	Sin transición definida para $\delta(q_0, 1)$. RECHAZA (q_r).

Ejecución de aceptación para $w_a = 01010$:

$C_0 = (q_0, \epsilon, 01010)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_1 = (q_1, 1, 1010)$	Aplica $\delta(q_1, 1) = (q_0, 0, \rightarrow)$
$C_2 = (q_0, 10, 010)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_3 = (q_1, 101, 10)$	Aplica $\delta(q_1, 1) = (q_0, 0, \rightarrow)$
$C_4 = (q_0, 1010, 0)$	Aplica $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
$C_5 = (q_1, 10101, \sqcup)$	Aplica $\delta(q_1, \sqcup) = (q_a, \sqcup, \leftarrow)$
$C_6 = (q_a, 1010, 1\sqcup)$	ACEPTA.

Complejidad de M : La máquina avanza de forma estrictamente lineal, procesando un símbolo por cada transición hacia la derecha sin realizar retrocesos durante la lectura. Al llegar al final de la cadena de longitud N , ejecuta una transición final para evaluar el $\sqcup$. El número de pasos es exactamente $N + 1$. Por lo tanto:

$$f_M(N) = N + 1 \implies \mathbf{f}_M(\mathbf{N}) \in \mathcal{O}(\mathbf{N})$$

Solución al Ejercicio 3.C

Lenguaje descrito:

$$L_M = \{0^n 1^m \mid n \geq 0, m \geq 1\}$$

La máquina permanece en q_0 borrando cualquier secuencia inicial (posiblemente vacía) de ceros. Requiere encontrar al menos un '1' para transitar a q_1 , donde permanece borrando

cualquier cantidad de '1's adicionales. Si, estando en q_1 , se encuentra con otro '0' (lo que indicaría un '0' después de un '1'), la máquina rechaza. Acepta si logra vaciar la cadena y encuentra el $\sqcup$ en el estado q_1 .

Consideramos las cadenas originales para ejecución: $w_r = 011011$ (rechazo) y $w_a = 0000111$ (aceptación).

Ejecución de rechazo para $w_r = 011011$:

$C_0 = (q_0, \epsilon, 011011)$	Aplica $\delta(q_0, 0) = (q_0, \sqcup, \rightarrow)$
$C_1 = (q_0, \sqcup, 11011)$	Aplica $\delta(q_0, 1) = (q_1, \sqcup, \rightarrow)$
$C_2 = (q_1, \sqcup\sqcup, 1011)$	Aplica $\delta(q_1, 1) = (q_1, \sqcup, \rightarrow)$
$C_3 = (q_1, \sqcup\sqcup\sqcup, 011)$	Sin transición definida para $\delta(q_1, 0)$. RECHAZA (q_r).

Ejecución de aceptación para $w_a = 0000111$:

$C_0 = (q_0, \epsilon, 0000111)$	Aplica $\delta(q_0, 0) = (q_0, \sqcup, \rightarrow)$
$C_1 = (q_0, \sqcup, 000111)$	Aplica $\delta(q_0, 0) = (q_0, \sqcup, \rightarrow)$
$C_2 = (q_0, \sqcup\sqcup, 00111)$	Aplica $\delta(q_0, 0) = (q_0, \sqcup, \rightarrow)$
$C_3 = (q_0, \sqcup\sqcup\sqcup, 0111)$	Aplica $\delta(q_0, 0) = (q_0, \sqcup, \rightarrow)$
$C_4 = (q_0, \sqcup\sqcup\sqcup\sqcup, 111)$	Aplica $\delta(q_0, 1) = (q_1, \sqcup, \rightarrow)$
$C_5 = (q_1, \sqcup\sqcup\sqcup\sqcup, 11)$	Aplica $\delta(q_1, 1) = (q_1, \sqcup, \rightarrow)$
$C_6 = (q_1, \sqcup\sqcup\sqcup\sqcup\sqcup, 1)$	Aplica $\delta(q_1, 1) = (q_1, \sqcup, \rightarrow)$
$C_7 = (q_1, \sqcup\sqcup\sqcup\sqcup\sqcup\sqcup, \sqcup)$	Aplica $\delta(q_1, \sqcup) = (q_a, \sqcup, \leftarrow)$
$C_8 = (q_a, \sqcup\sqcup\sqcup\sqcup\sqcup\sqcup, \sqcup\sqcup)$	ACEPTA.

Complejidad de M : Similar al inciso anterior, el cabezal avanza en una sola dirección borrando un símbolo por cada transición, ya sea en q_0 o q_1 . Para una cadena válida de longitud N , el cabezal realiza N desplazamientos a la derecha, más 1 transición final al leer $\sqcup$. El número de pasos es $N + 1$. Por lo tanto:

$$f_M(N) = N + 1 \implies \mathbf{f}_M(\mathbf{N}) \in \mathcal{O}(\mathbf{N})$$

Ejercicio 4

Enunciado

Investiga qué son las funciones de tiempo "construibles" (Time constructible functions).

1. Elige dos de las siguientes funciones y demuestra que son de tiempo construibles:
 - (a) $2n \log n$
 - (b) $3n^2$
 - (c) 3^n
2. Da otros ejemplos (diferentes a los del inciso anterior) de funciones de tiempo construible.
3. Da un par de ejemplos de funciones que no son de tiempo construible. Justifica por qué no lo son.

Solución

Las funciones de tiempo construibles se refiere a una función $f(n)$ tal que existe una máquina de Turing determinista de múltiples cintas que se detiene en $f(n)$ cuando se le da una palabra de entrada de longitud n . Dichas funciones desempeñan un papel importante en la teoría de la computación. Son interesantes por sí mismas como ejemplos de funciones muy 'honestas'. Por lo tanto, es deseable contar con técnicas potentes para demostrar la construibilidad en tiempo de las funciones.

Por máquina de Turing nos referimos a una máquina de Turing determinista de múltiples cintas. Se supone que todas las cintas son cintas de trabajo (es decir, no utilizamos cintas especiales de entrada o salida) y que las cintas son infinitas en ambas direcciones. Generalmente $\mathbb{N}$ denota el conjunto de todos los enteros no negativos. Se dice que una función de d variables $f(n_1, \dots, n_d)$ de $\mathbb{N}^d$ a $\mathbb{N}$ es *construible en tiempo* (*ct* para abreviar) si existe una máquina de Turing con al menos d cintas que se detiene en el tiempo $f(n_1, \dots, n_d)$ cuando se inicia con la siguiente configuración en el tiempo 0: (1) para cada i ($1 \leq i \leq d$), el contenido de la i -ésima cinta es $\dots BB1^{n_i}BB\dots$ y el cabezal escanea el 1 situado más a la izquierda (si $n_i > 0$) (B denota el símbolo blanco y 1^{n_i} denota la secuencia de n_i unos), y (2) el contenido de las otras cintas, si las hay, está completamente en blanco.[1]

- Elige dos de las siguientes funciones y demuestra que son de tiempo construibles:
 - a. $2n \log n$
 - b. $3n^2$
 - c. 3^n

Tomando el **inciso b)**:

Sea $3n^2$ decimos es una función en tiempo construible si existe una máquina MT de múltiples cintas, con la entrada 1^{n_i} (representación unaria de n), produce la representación binaria de $f(n)$ en $O(f(n))$ pasos. Se puede simular M de la siguiente forma.

Se necesita escanear la cinta de entrada que contiene 1^n para contar el número de caracteres, convirtiéndolo en un entero binario n en una segunda cinta. Después se tiene que calcular n^2 . Para esto el entero binario n por sí mismo utilizando un algoritmo estándar de multiplicación binaria para obtener n^2 . Multiplicar n^2 por 3 (lo cual equivale simplemente a desplazar n^2 un bit a la izquierda para obtener $2n^2$ y sumarlo a n^2 , o sea $3n^2 = 2n^2 + n^2$).

Convertir 1^n a binario requiere exactamente n pasos. La representación binaria de n tiene $\lfloor \log_2 n \rfloor + 1$ bits. Multiplicar dos números de m bits requiere $O(m^2)$ pasos. Si tenemos, $m = O(\log n)$, elevar n al cuadrado requiere $O(\log^2 n)$ pasos. Multiplicar por 3 conlleva un tiempo lineal respecto al número de bits de n^2 , es decir, $O(\log n)$ pasos.

Sumando estos valores, el tiempo total $T(n)$ necesario para calcular la salida binaria es: $T(n) = O(n) + O(\log^2 n) + O(\log n) = O(n)$. Dado que $O(n) \leq O(3n^2)$ para todo $n \geq 1$, así $3n^2$ es construible en tiempo.

Tomando el **inciso c)** consideramos:

Sea 3^n decimos es una función en tiempo construible si existe una máquina MT de múltiples cintas, con la entrada 1^{n_i} (representación unaria de n), produce la representación binaria de $f(n)$ en $O(f(n))$ pasos. Se puede simular M de la siguiente forma.

Escribir el número binario $1(3^0)$ en la Cinta 2. Mantener la entrada 1^n en la Cinta 1 para que actúe como contador del bucle. Recorrer la entrada de izquierda a derecha, y por cada 1 leído, multiplicar por 3 el numeral de la zona de trabajo. Al encontrar el primer $\sqcup$, detenerse.

Multiplicar por 3 en binario es una sola operación compuesta ($3x = 2x + x$). Es decir, que se debe de recorrer un lugar y sumar el original. Después de tachar todos los n símbolos de la Cinta 1, la Cinta 2 contiene la representación binaria exacta de 3^n .

Ahora podemos analizar el número total de operaciones realizadas por M , para la Longitud en bits tras i iteraciones, el valor en la Cinta 2 es 3^i , por lo que el número de bits necesario para almacenar 3^i es $\lfloor \log_2(3^i) \rfloor + 1 \approx i \cdot \log_2(3) \approx 1,58i$.

En la i -ésima iteración, copiar, desplazar y sumar un número binario de longitud $O(i)$ requiere $O(i)$ pasos. Sumar los pasos de todas las n iteraciones da una serie aritmética: $\sum_{i=1}^n O(i) = O\left(\frac{n(n+1)}{2}\right) = O(n^2)$.

Dado que $O(n^2) \leq O(3^n)$ para todo $n \geq 1$, la máquina de Turing calcula 3^n . Por lo tanto, 3^n es una función construible en tiempo.

- Da otros ejemplos (diferentes a los del inciso anterior) de funciones de tiempo construible.

Muchas funciones simples como n^2 , n^3 , 2^n , $n!$ son ct. Las demostraciones no son fundamentalmente difíciles, pero sí tediosas.

Para n^3 , sucede un caso similar a n^2 . Suponemos a M determinista de 3 cintas de trabajo, donde cada una contiene inicialmente la entrada unaria 1^n . La máquina ejecuta un esquema de tres bucles anidados. La cinta 1 actúa como contador externo y realiza n iteraciones. Para la cinta 2 realiza un recorrido completo de n posiciones por cada celda procesada en la cinta 1 (n^2 iteraciones en total). Así para la cinta 3 ejecuta un barrido completo de n celdas (n pasos) por cada movimiento de la Cinta 2.

El número total de transiciones ejecutadas hasta la parada es $T(n) = n \cdot n \cdot n = n^3$. La máquina se detiene en exactamente n^3 pasos, $f(n) = n^3$ es construible en tiempo.

Para $f(n) = n!$, se puede suponer una máquina M que utiliza la cadena de entrada 1^n para controlar las llamadas anidadas de n niveles de profundidad.

En la m -ésima iteración del bucle, multiplicamos $(m-1)!$ por m . El número de bits necesarios para representar $(m-1)!$ es $O(m \log m)$, y el número de bits para m es $O(\log m)$, multiplicarlos sería $O((m \log m) \cdot \log m) = O(m \log^2 m)$ pasos.

Sumamos los costos $T(n) = \sum_{m=1}^n O(m \log^2 m) = O(n^2 \log^2 n)$, entonces $f(n) = n!$ es construible en tiempo.[2]

- Da un par de ejemplos de funciones que no son de tiempo construible. Justifica por qué no lo son.

La función $f(n) = \log n$

Para cualquier función construible en tiempo bajo la definición estándar, la máquina de Turing debe al menos leer la cadena de entrada 1^n de longitud n .

Supongamos que la cadena de entrada en la cinta de la máquina de Turing es 1^{1000} , de acuerdo con la función $f(n) = \log_2 n$, la máquina debería finalizar su cómputo y detenerse aproximadamente:

$$f(1000) = \log_2(1000) = 9 \text{ pasos}$$

Para que la máquina de Turing pueda procesar o reconocer que el tamaño de la entrada es $n = 1000$, el cabezal debe desplazarse celda por celda a lo largo de la cinta, por lo que leerla requiere un mínimo indispensable de 1000 pasos, solo que al llegar al paso 10, el cabezal solo ha logrado avanzar sobre los primeros 10 símbolos (1^{10}). Quedan aún 990 símbolos de la entrada sin leer.

Leer una entrada de longitud n requiere un mínimo de n pasos físicos de desplazamiento del cabezal, como $\log n = o(n)$, la función crece demasiado lento: la máquina agota el presupuesto de tiempo permitido ($\log n$ pasos) antes de haber podido leer la entrada completa.

Teorema de Jerarquía de Tiempo. Aquí fungen funciones construidas mediante saltos condicionales basados en el problema de la parada o usando la diagonalización del Teorema de Jerarquía de Tiempo

Para entender por qué no es construible en tiempo, debemos examinar cómo está construida mediante una paradoja intencionada (autorreferencia y diagonalización).

Sea $M_1, M_2, M_3, \dots$ una enumeración de todas las máquinas de Turing posibles. Definimos artificialmente una función $T(n)$ paso a paso para cada tamaño de entrada n . Tomamos la n -ésima máquina de Turing, M_n , y le damos la entrada n .

Observamos el comportamiento de $M_n(n)$, por ejemplo, definir $f(n)$ de modo que valga $2n$ si una máquina M_n se detiene en su propia entrada, y $2n + 1$ si no lo hace (o variaciones similares).

No es constructible porque si pudieras construir esta función eficientemente en tiempo $O(f(n))$, podrías usar ese algoritmo para resolver o decidir problemas indecidibles (como el problema de la parada), lo cual genera una contradicción.[3]

Referencias

- [1] Kobayashi, K. (1985). On proving time constructibility of functions. *Theoretical Computer Science*, 35, 215–225. Disponible en: [https://doi.org/10.1016/0304-3975\(85\)90015-5](https://doi.org/10.1016/0304-3975(85)90015-5).
- [2] Blais, E. (2026). *CS 365: Models of Computation – Time Complexity* [Notas de clase]. David R. Cheriton School of Computer Science, University of Waterloo. Disponible en: <https://cs.uwaterloo.ca/~eblais/cs365/w26/time>.
- [3] Mathematics Stack Exchange. (2014). *Why are the hierarchy theorem proofs called diagonalization?* Disponible en: <https://math.stackexchange.com/questions/658854/why-are-the-hierarchy-theorem-proofs-called-diagonalization>.

Referencias Bibliográficas

Referencias

- [1] Kobayashi, K. (1985). On proving time constructibility of functions. *Theoretical Computer Science*, 35, 215–225. [https://doi.org/10.1016/0304-3975\(85\)90015-5](https://doi.org/10.1016/0304-3975(85)90015-5)
- [2] Blais, E. (2026). *CS 365: Models of Computation – Time Complexity* [Notas de clase]. David R. Cheriton School of Computer Science, University of Waterloo. <https://cs.uwaterloo.ca/~eblais/cs365/w26/time>
- [3] Mathematics Stack Exchange. (2014). *Why are the hierarchy theorem proofs called diagonalization?* <https://math.stackexchange.com/questions/658854/why-are-the-hierarchy-theorem-proofs-called-diagonalization>